MOBILE APPLICATION DEVELOPMENT USING ANDROID

ASSIGNMENT 4 — SOLVED ANSWERS

Name: Shreya Paliwal **Course:** BCA

Note: The answers below use Android concepts and Java-based examples. Some Android APIs have newer alternatives; the examples are intended to explain the assignment concepts clearly.

Q1. Explain the various User Interface Screen Elements used in Android application development, with suitable examples.

Android user interface (UI) screen elements are the visual components through which users interact with an application. They are generally created in XML layout files and controlled from Java or Kotlin code.

Important UI elements

1. TextView: Displays read-only text, such as a heading or label. Example: `android:text="Welcome"`.

2. EditText: Accepts user input, such as a name, email address or password. The input type can be specified using `android:inputType`.

3. Button: Performs an action when tapped, such as submitting a form or opening another screen.

4. ImageView: Displays an image or icon from drawable resources.

5. CheckBox: Allows the user to select more than one option.

6. RadioButton and RadioGroup: Allow the user to select one option from a group.

7. Switch: Represents an on/off setting, such as enabling notifications.

8. Spinner: Shows a drop-down list from which the user selects one item.

9. ProgressBar: Indicates that an operation is in progress.

10. ListView/RecyclerView: Displays a scrollable collection of items, such as student records.

Example XML

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Student Registration" />

    <EditText
        android:id="@+id/nameInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter name" />

    <Button
        android:id="@+id/submitButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Submit" />
</LinearLayout>
```

These elements can be combined to create forms, menus, dashboards and other interactive screens. Good UI design should be simple, consistent, readable and accessible.

Q2. Discuss the process of designing user interfaces using Layouts. Compare Linear Layout, Relative Layout, and Constraint Layout with examples.

A layout is a ViewGroup that controls the position and size of UI elements on an Android screen. To design an interface, a developer identifies the required controls, selects a suitable parent layout, places the controls inside it, sets dimensions and spacing, and tests the screen on different device sizes.

Types of layouts

LinearLayout: Arranges child views in a single row or a single column. The direction is set with `android:orientation`. It is simple for forms and vertically stacked controls.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Name" />
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />
</LinearLayout>
```

RelativeLayout: Positions views relative to the parent or to other views, using rules such as below, above, alignParentEnd and centerInParent. It can reduce nested layouts, but complex rules may be harder to maintain.

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
    <TextView
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@id/title"
        android:text="Continue" />
</RelativeLayout>
```

ConstraintLayout: Positions views by connecting their edges to the parent or to other views through constraints. It supports flexible, responsive screens and helps avoid deeply nested layouts. It is commonly used with Android Studio's Layout Editor.

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Comparison: LinearLayout is best for a simple row or column; RelativeLayout is useful for rule-based positioning; ConstraintLayout is suitable for complex and responsive screens. The choice depends on screen complexity and maintainability.

Q3. What is meant by Drawing in Android? Explain how custom views can be created and drawn using the Canvas and Paint classes.

Drawing in Android means creating graphics such as lines, circles, shapes, text and custom illustrations on a screen. A developer can create a custom View by extending the View class and overriding its onDraw(Canvas) method.

Canvas provides drawing methods such as drawLine(), drawRect(), drawCircle() and drawText(). **Paint** defines the appearance of the drawing, including colour, stroke width, text size and fill or stroke style.

Example: custom view that draws a circle

```
public class CircleView extends View {
    private Paint paint = new Paint(Paint.ANTI_ALIAS_FLAG);

    public CircleView(Context context) {
        super(context);
        paint.setColor(Color.BLUE);
        paint.setStyle(Paint.Style.FILL);
    }

    @Override
    protected void onDraw(Canvas canvas) {
        super.onDraw(canvas);
        float cx = getWidth() / 2f;
        float cy = getHeight() / 2f;
        canvas.drawCircle(cx, cy, 80, paint);
    }
}
```

To use the custom view, create an instance in an Activity or include its fully qualified class name in an XML layout. Android calls onDraw() when the view needs to be displayed. Call invalidate() when the drawing must be refreshed. Drawing should be kept efficient because onDraw() may be called frequently.

Q4. Explain the concept of Animation in Android applications. Discuss the different types of animations (View Animation, Property Animation, Drawable Animation) with examples.

Animation creates the appearance of movement or change in an Android application. It improves user experience by providing visual feedback and making transitions smoother.

1. View Animation (Tween Animation): Applies visual transformations such as alpha (fade), scale, rotate and translate to a View. It changes how the view is drawn, rather than changing all of its underlying properties.

```
// res/anim/fade_in.xml
<alpha xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromAlpha="0.0"
    android:toAlpha="1.0"
    android:duration="1000" />

// In an Activity
Animation animation =
    AnimationUtils.loadAnimation(this, R.anim.fade_in);
myView.startAnimation(animation);
```

2. Property Animation: Changes an object's actual property values over time, such as alpha, rotation, translation or scale. ObjectAnimator and ValueAnimator are commonly used.

```
ObjectAnimator animator =
    ObjectAnimator.ofFloat(myView, "translationX", 0f, 300f);
animator.setDuration(1000);
animator.start();
```

3. Drawable Animation (Frame Animation): Displays a sequence of drawable images one after another, like a flipbook. Frames are defined in an animation-list drawable resource.

```
// res/drawable/frames.xml
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="true">
    <item android:drawable="@drawable/frame1" android:duration="150" />
    <item android:drawable="@drawable/frame2" android:duration="150" />
</animation-list>

// In an Activity, after setting frames.xml on an ImageView:
AnimationDrawable frames =
    (AnimationDrawable) imageView.getDrawable();
frames.start();
```

View animation is useful for simple visual effects, property animation for changing object properties, and drawable animation for frame-by-frame image sequences.

Q5. Explain Android Data and Storage APIs. Discuss the various options available for storing data in an Android application (SharedPreferences, Internal Storage, External Storage, SQLite).

Android provides different storage APIs according to the amount, structure and sharing requirements of data.

1. SharedPreferences: Stores small amounts of simple key-value data, such as theme choice or login setting. It is suitable for preferences, not large records. Modern Android projects may also use DataStore.

```
SharedPreferences prefs =
    getSharedPreferences("app_settings", MODE_PRIVATE);
prefs.edit().putString("username", "Shreya").apply();

String username = prefs.getString("username", "Guest");
```

2. Internal Storage: Stores files in the app's private internal directory. Other ordinary apps cannot access these files. They are removed when the app is uninstalled.

```
try (FileOutputStream out =
         openFileOutput("note.txt", MODE_PRIVATE)) {
    out.write("Hello Android".getBytes());
} catch (IOException e) {
    e.printStackTrace();
}
```

3. External Storage: Stores files in shared or app-specific external storage. Shared media and documents may require appropriate user selection or permissions depending on Android version and storage location. App-specific external files are normally removed on uninstall.

4. SQLite: A lightweight relational database available on Android. It stores structured data in tables and supports SQL operations such as INSERT, SELECT, UPDATE and DELETE. Room is the recommended abstraction for many modern applications.

Selection: Use SharedPreferences/DataStore for small settings, internal files for private files, external storage for user-facing/shared files, and SQLite/Room for structured records.

Q6. What is SQLite? Explain, with the help of code snippets, how to create a database, insert records, and query data in an Android application.

SQLite is a lightweight, serverless relational database engine. It stores data locally in a database file and does not require a separate database server. Android provides SQLiteOpenHelper to help create and upgrade databases.

Create a helper class

```
public class DBHelper extends SQLiteOpenHelper {
    private static final String DB_NAME = "College.db";
    private static final int DB_VERSION = 1;

    public DBHelper(Context context) {
        super(context, DB_NAME, null, DB_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE Student (" +
            "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
            "name TEXT, course TEXT, marks REAL)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db,
                          int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS Student");
        onCreate(db);
    }

    public boolean insertStudent(String name,
                                 String course, double marks) {
        SQLiteDatabase db = getWritableDatabase();
        ContentValues values = new ContentValues();
        values.put("name", name);
        values.put("course", course);
```

```
        values.put("marks", marks);
        long result = db.insert("Student", null, values);
        return result != -1;
    }

    public Cursor getAllStudents() {
        SQLiteDatabase db = getReadableDatabase();
        return db.rawQuery(
            "SELECT id, name, course, marks FROM Student",
            null);
    }
}
```

Use the helper in an Activity

```
DBHelper helper = new DBHelper(this);

// Insert one record
boolean inserted =
    helper.insertStudent("Aman", "BCA", 85.5);

// Read records
Cursor cursor = helper.getAllStudents();
while (cursor.moveToNext()) {
    String name = cursor.getString(
        cursor.getColumnIndexOrThrow("name"));
    String course = cursor.getString(
        cursor.getColumnIndexOrThrow("course"));
    double marks = cursor.getDouble(
        cursor.getColumnIndexOrThrow("marks"));
    // Add these values to the list shown in the UI.
}
cursor.close();
```

The database is created when it is first accessed. insert() adds a row and returns its row ID (or -1 on failure). A Cursor provides access to query results and must be closed after use. For production applications, perform database work away from the main UI thread and prefer parameterized queries or Room.

Q7. Explain the concept of Content Providers in Android. Discuss how data can be shared between different applications using Content Providers.

A Content Provider is an Android component that manages and exposes application data in a standard way. It allows other applications to access data through a content URI, subject to permissions. Examples include providers for contacts and media.

A Content Provider commonly implements query(), insert(), update(), delete() and getType(). Data is identified using a content URI, such as `content://authority/table`. A ContentResolver is used by a client application to communicate with a provider.

Basic steps

1. The provider application creates a class extending ContentProvider. 2. It defines a unique authority and URI patterns. 3. It implements CRUD methods and validates incoming requests. 4. It declares the provider in AndroidManifest.xml and applies suitable read/write permissions. 5. The client obtains a ContentResolver and uses the provider's URI to access permitted data.

```
// Client-side example: query a provider URI
Uri uri = Uri.parse(
    "content://com.example.provider/students");

Cursor cursor = getContentResolver().query(
    uri,
    new String[]{"_id", "name"},
    null, null, "name ASC");

if (cursor != null) {
    while (cursor.moveToNext()) {
        String name = cursor.getString(
            cursor.getColumnIndexOrThrow("name"));
    }
    cursor.close();
}
```

Content Providers provide controlled data sharing, a common interface for CRUD operations and permission-based access. Applications should expose only the data required and protect sensitive information.

Q8. Explain Android Networking APIs. Discuss the steps involved in sending an HTTP request from an Android application and parsing the JSON response received.

Android networking APIs allow an application to communicate with web servers and online services. Common choices include HttpURLConnection for basic HTTP communication and libraries such as Retrofit or OkHttp for larger projects.

Steps

1. Add the INTERNET permission in AndroidManifest.xml. 2. Build the service URL and request. 3. Execute the network operation on a background thread. 4. Read the response and check its status code. 5. Parse JSON using JSONObject/JSONArray or a JSON library. 6. Update the UI on the main thread. Handle timeouts, errors and unavailable networks.

```
// AndroidManifest.xml
<uses-permission android:name="android.permission.INTERNET" />

// Simplified Java example; run this in a background thread
URL url = new URL("https://example.com/api/student");
HttpURLConnection connection =
    (HttpURLConnection) url.openConnection();
connection.setRequestMethod("GET");
connection.setConnectTimeout(10000);
connection.setReadTimeout(10000);

int status = connection.getResponseCode();
InputStream stream = (status >= 200 && status < 300)
    ? connection.getInputStream()
    : connection.getErrorStream();

StringBuilder response = new StringBuilder();
try (BufferedReader reader = new BufferedReader(
        new InputStreamReader(stream))) {
    String line;
    while ((line = reader.readLine()) != null) {
        response.append(line);
    }
}
connection.disconnect();

// Parse JSON response
JSONObject object = new JSONObject(response.toString());
String studentName = object.getString("name");
```

The example assumes the server returns a JSON object containing a name field. Real applications should validate the response, handle exceptions, avoid blocking the main thread and use HTTPS.

Q9. Compare Android Web APIs and Android Telephony APIs. Explain, with examples, how each is used in mobile application development.

Android Web APIs are used to display web content or communicate with web services. Examples include WebView for showing a web page inside an app and HTTP networking APIs for exchanging data with a server. They are useful for online forms, dashboards, maps and REST-based applications.

```
// Display a web page in a WebView
WebView webView = findViewById(R.id.webView);
webView.getSettings().setJavaScriptEnabled(false);
webView.loadUrl("https://www.example.com");
```

Android Telephony APIs provide information about telephony services and, where permitted, access to phone-related features. TelephonyManager can provide information such as network or SIM state, subject to Android version, permissions and privacy restrictions. Apps can open the dialler using an ACTION_DIAL intent.

```
// Open the phone dialler with a number filled in
Intent intent = new Intent(
    Intent.ACTION_DIAL,
    Uri.parse("tel:9876543210"));
```

```
startActivity(intent);
```

Comparison: Web APIs focus on web pages and internet-based communication; Telephony APIs focus on mobile network and phone functionality. Web APIs are used in online shopping or news apps, while Telephony-related features may be used in dialler, communication or network-status applications. Access to sensitive telephony information is restricted and should not be assumed to be available.

Q10. Coding Question: Create a simple Android application that uses SQLite to store student records (Name, Course, Marks). The app should allow inserting a new record and displaying all records in a ListView/RecyclerView. Also briefly explain the steps required to deploy this application to the Google Play Store.

The following basic Java implementation uses SQLiteOpenHelper and a ListView. Create an Android Studio project and add the XML layout and Java classes below. This is a compact teaching example; add input validation and background database work before production use.

1. activity_main.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Student name" />

    <EditText
        android:id="@+id/course"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Course" />

    <EditText
        android:id="@+id/marks"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Marks"
        android:inputType="numberDecimal" />

    <Button
        android:id="@+id/save"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Save Student" />

    <ListView
        android:id="@+id/studentList"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />
</LinearLayout>
```

2. StudentDB.java

```
public class StudentDB extends SQLiteOpenHelper {
    public StudentDB(Context context) {
        super(context, "StudentDB", null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE students (" +
            "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
            "name TEXT NOT NULL, course TEXT NOT NULL, " +
            "marks REAL NOT NULL)");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db,
                          int oldVersion, int newVersion) {
```

```
        db.execSQL("DROP TABLE IF EXISTS students");
        onCreate(db);
    }

    public boolean addStudent(String name,
                              String course, double marks) {
        ContentValues cv = new ContentValues();
        cv.put("name", name);
        cv.put("course", course);
        cv.put("marks", marks);
        return getWritableDatabase().insert(
            "students", null, cv) != -1;
    }

    public ArrayList<String> getStudents() {
        ArrayList<String> list = new ArrayList<>();
        Cursor c = getReadableDatabase().rawQuery(
            "SELECT name, course, marks FROM students",
            null);
        while (c.moveToNext()) {
            list.add(c.getString(0) + " | " +
                c.getString(1) + " | Marks: " +
                c.getDouble(2));
        }
        c.close();
        return list;
    }
}
```

3. MainActivity.java

```
public class MainActivity extends AppCompatActivity {
    EditText name, course, marks;
    ListView studentList;
    StudentDB db;
    ArrayAdapter<String> adapter;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        name = findViewById(R.id.name);
        course = findViewById(R.id.course);
        marks = findViewById(R.id.marks);
        studentList = findViewById(R.id.studentList);
        Button save = findViewById(R.id.save);

        db = new StudentDB(this);
        adapter = new ArrayAdapter<>(this,
            android.R.layout.simple_list_item_1,
            db.getStudents());
        studentList.setAdapter(adapter);

        save.setOnClickListener(v -> {
            String n = name.getText().toString().trim();
            String co = course.getText().toString().trim();
            String m = marks.getText().toString().trim();

            if (n.isEmpty() || co.isEmpty() || m.isEmpty()) {
                Toast.makeText(this, "Enter all details",
                    Toast.LENGTH_SHORT).show();
                return;
            }

            try {
                double value = Double.parseDouble(m);
                if (value < 0 || value > 100) {
                    Toast.makeText(this, "Marks must be 0-100",
                        Toast.LENGTH_SHORT).show();
                    return;
                }

                if (db.addStudent(n, co, value)) {
                    name.setText("");
                    course.setText("");
                    marks.setText("");
                    adapter.clear();
                    adapter.addAll(db.getStudents());
                    adapter.notifyDataSetChanged();
```

```
                    Toast.makeText(this, "Student saved",
                        Toast.LENGTH_SHORT).show();
                }
            } catch (NumberFormatException e) {
                Toast.makeText(this, "Enter valid marks",
                    Toast.LENGTH_SHORT).show();
            }
        });
    }
}
```

How the application works: The user enters a student's name, course and marks. On tapping Save Student, the app validates the fields and inserts the record into the SQLite students table. It then reloads the database records into an ArrayAdapter, which displays them in the ListView.

4. Steps to deploy the application on Google Play Store

1. Create a Google Play Console developer account and complete the required registration. 2. Prepare the app name, icon, screenshots, description and privacy policy where required. 3. Set the application ID/package name and version information. 4. Generate a signed Android App Bundle (AAB) using Android Studio. 5. Create the app in Play Console and upload the AAB to a testing track. 6. Complete the store listing, app content, data safety and target audience declarations accurately. 7. Test the release, resolve warnings and submit it for review. 8. After approval, publish the app and monitor crashes, reviews and updates.

Google Play requirements can change. Developers must follow the current Play Console policies, target API requirements and testing requirements shown in their account.

— End of Assignment 4 —